



Experiment No. 1

Title: Implementation of Tokenization with different python libraries



Batch: NLP-1

Experiment No.: 1

Roll No.: 16010422234

Name: Chandana Galgali

Aim of the Experiment: To implement Tokenization using different python libraries

Program/Steps:

1. Apply tokenization using different python libraries with different text inputs such as tweets, news articles and so on.
2. Write your observations regarding using different methods of tokenization with different input text data.

```
import re
import nltk
from textblob import TextBlob
from gensim.utils import simple_preprocess
from tensorflow.keras.preprocessing.text import text_to_word_sequence
from nltk.tokenize import (
    word_tokenize, sent_tokenize, RegexpTokenizer,
    TreebankWordTokenizer, TweetTokenizer, MWETokenizer
)
import spacy
import en_core_web_sm

# =====
# Setup
# =====
nltk.download('punkt', quiet=True)
nlp = en_core_web_sm.load()

# =====
# Sample Texts
# =====
texts = {
    "Tweet": """OMG 🤯 New phone drop!!! Check this out: https://t.co/abcd
@realUser can't wait 😂🔥 #TechNews #NewLaunch""",
    "News": (
        "The Reserve Bank raised interest rates by 25 basis points on
Tuesday, "
        "citing persistent inflation. Analysts expect a cautious outlook
through Q4 2025."
    ),
}
```

```

    "General": (
        "But I'm glad you'll see me as I am. Above all, I wouldn't want
people to think "
        "that I want to prove anything. I just want to live; to cause no
evil to anyone "
        "but myself. I have that right, haven't I? — Leo Tolstoy"
    )
}

# =====
# 1) Python split & regex
# =====
def py_split_tokens(text):
    return text.split()

def regex_word_tokens(text, pattern=r"\w+"):
    return re.findall(pattern, text)

regex_social = r"(https?:\/\/\S+) | (@\w+) | (#\w+) | (\w+' \w+) | (\w+' \w+) | (\w+)"

def regex_social_tokens(text):
    raw = re.findall(regex_social, text)
    return [next(g for g in m if g) for m in raw]

# =====
# 2) NLTK tokenizers
# =====
tb = TreebankWordTokenizer()
tt = TweetTokenizer(strip_handles=False, reduce_len=True,
preserve_case=False)
regexp_tokenizer = RegexpTokenizer(r"\w+")
mwe = MWETokenizer([("New", "York"), ("machine", "learning"), ("interest",
"rates")], separator="_")

def mwe_tokens(text):
    return mwe.tokenize(word_tokenize(text))

# =====
# 3) TextBlob
# =====
def textblob_tokens(text):
    blob = TextBlob(text)
    return [str(s) for s in blob.sentences], blob.words

```

```

# =====
# 4) spaCy
# =====
def spacy_tokens(text):
    doc = nlp(text)
    return [t.text for t in doc]

def spacy_sentences(text):
    doc = nlp(text)
    return [s.text for s in doc.sents]

# =====
# 5) Gensim
# =====
def gensim_tokens(text):
    return simple_preprocess(text)

# =====
# 6) Keras
# =====
def keras_tokens(text):
    return text_to_word_sequence(text)

# =====
# 7) RegexpTokenizer Examples
# =====
def regexp_examples():
    text = "The price of the car is 50,000 dollars. Model-X isn't cheap!"
    print("\nRegexpTokenizer (\\w+):",
    RegexpTokenizer(r"\\w+").tokenize(text))
    print("RegexpTokenizer (keep apostrophes):",
    RegexpTokenizer(r"[A-Za-z]+(?:'[A-Za-z]+)?" ).tokenize(text))
    print("RegexpTokenizer (numbers with commas/decimals):",
    RegexpTokenizer(r"\d[\\d,\\.]*").tokenize(text))

# =====
# 8) Observations
# =====
def observe(name, txt):
    wb = word_tokenize(txt)
    tw = TweetTokenizer(preserve_case=False).tokenize(txt)
    sp = [t.text for t in nlp(txt)]
    gs = simple_preprocess(txt)
    kz = text_to_word_sequence(txt)

```

```

print(f"\n=== Observations: {name} ===")
print("NLTK word_tokenize:", wb[:10])
print("TweetTokenizer:", tw[:10])
print("spaCy:", sp[:10])
print("Gensim:", gs[:10])
print("Keras:", kz[:10])

# =====
# Main execution
# =====
if __name__ == "__main__":
    for name, txt in texts.items():
        print(f"\n==== {name} ====")
        print("--- Python split:", py_split_tokens(txt))
        print("--- Regex \\w+:", regex_word_tokens(txt))
        print("--- Regex social-aware:", regex_social_tokens(txt))
        print("\nNLTK:")
        print("Sentence:", sent_tokenize(txt))
        print("Word:", word_tokenize(txt))
        print("Treebank:", tb.tokenize(txt))
        print("Tweet:", tt.tokenize(txt))
        print("Regexp:", regexp_tokenizer.tokenize(txt))
        print("MWE:", mwe_tokens(txt))
        print("\nTextBlob:")
        sents, words = textblob_tokens(txt)
        print("Sentences:", sents)
        print("Words:", words)
        print("\nspaCy:")
        print("Sentences:", spacy_sentences(txt))
        print("Tokens:", spacy_tokens(txt))
        print("\nGensim:", gensim_tokens(txt))
        print("Keras:", keras_tokens(txt))
        observe(name, txt)
    regexp_examples()

```

Output/Result:

```

===== Tweet =====
--- Python split: ['OMG', '🔥', 'New', 'phone', 'drop!!!', 'Check', 'this', 'out:', 'https://t.co/abcd', '@realUser', 'can't', 'wait', '😬', '#TechNews', '#NewLaunch']
--- Regex \w+: ['OMG', 'New', 'phone', 'drop', 'Check', 'this', 'out', 'https', 't', 'co', 'abcd', 'realUser', 'can', 't', 'wait', 'TechNews', 'NewLaunch']
--- Regex social-aware: ['OMG', 'New', 'phone', 'drop', 'Check', 'this', 'out', 'https://t.co/abcd', '@realUser', 'can't', 'wait', '#TechNews', '#NewLaunch']

NLTK:
Sentence: ['OMG', '🔥', 'New phone drop!!!', 'Check this out: https://t.co/abcd\n@realUser can't wait 😬 #TechNews #NewLaunch']
Word: ['OMG', '🔥', 'New', 'phone', 'drop', '!', '!', '!', 'Check', 'this', 'out', ':', 'https', 't', 'co', 'abcd', 'realUser', 'can', 't', 'wait', 'TechNews', 'NewLaunch']
Treebank: ['OMG', '🔥', 'New', 'phone', 'drop', '!', '!', '!', 'Check', 'this', 'out', ':', 'https', 't', 'co', 'abcd', '@', 'realUser', 'can't', 'wait', '😬', '#', 'TechNews', '#', 'NewLaunch']
Tweet: ['omg', '🔥', 'new', 'phone', 'drop', '!', '!', '!', 'check', 'this', 'out', ':', 'https://t.co/abcd', '@realuser', 'can', 't', 'wait', '😬', '#', 'technews', '#newlaunch']
Regex: ['OMG', 'New', 'phone', 'drop', 'Check', 'this', 'out', 'https', 't', 'co', 'abcd', 'realUser', 'can', 't', 'wait', 'TechNews', 'NewLaunch']
MWE: ['OMG', '🔥', 'New', 'phone', 'drop', '!', '!', '!', 'Check', 'this', 'out', ':', 'https', 't', 'co', 'abcd', '@', 'realUser', 'can', 't', 'wait', '😬', '#', 'TechNews', '#', 'NewLaunch']

TextBlob:
Sentences: ['OMG 🔥 New phone drop!!!', 'Check this out: https://t.co/abcd\n@realUser can't wait 😬 #TechNews #NewLaunch']
Words: ['OMG', '🔥', 'New', 'phone', 'drop', 'Check', 'this', 'out', 'https', 't.co/abcd', 'realUser', 'can', 't', 'wait', '😬', 'TechNews', 'NewLaunch']

spaCy:
Sentences: ['OMG 🔥 New phone drop!!!', 'Check this out: https://t.co/abcd\n@realUser can't wait 😬 #TechNews #NewLaunch']
Tokens: ['OMG', '🔥', 'New', 'phone', 'drop', '!', '!', '!', 'Check', 'this', 'out', ':', 'https://t.co/abcd', '@', 'realUser', 'can', 't', 'wait', '😬', '#', 'TechNews', '#', 'NewLaunch']

Gensim: ['omg', 'new', 'phone', 'drop', 'check', 'this', 'out', 'https', 'co', 'abcd', 'realuser', 'can', 'wait', 'technews', 'newlaunch']
Keras: ['omg', '🔥', 'new', 'phone', 'drop', 'check', 'this', 'out', 'https', 't', 'co', 'abcd', 'realuser', 'can't', 'wait', '😬', 'technews', 'newlaunch']

```

```

=== Observations: Tweet ===
NLTK word_tokenize: ['OMG', '🔥', 'New', 'phone', 'drop', '!', '!', '!', 'Check', 'this']
TweetTokenizer: ['omg', '🔥', 'new', 'phone', 'drop', '!', '!', '!', 'check', 'this']
spaCy: ['OMG', '🔥', 'New', 'phone', 'drop', '!', '!', '!', 'Check', 'this']
Gensim: ['omg', 'new', 'phone', 'drop', 'check', 'this', 'out', 'https', 'co', 'abcd']
Keras: ['omg', '🔥', 'new', 'phone', 'drop', 'check', 'this', 'out', 'https', 't']

```

```

===== News =====
--- Python split: ['The', 'Reserve', 'Bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points', 'on', 'Tuesday', 'citing', 'persistent', 'inflation.', 'Analysts', 'expect', 'a', 'cautious', 'outlook', 'through', 'Q4', '2025.']
--- Regex \w+: ['The', 'Reserve', 'Bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points', 'on', 'Tuesday', 'citing', 'persistent', 'inflation', 'Analysts', 'expect', 'a', 'cautious', 'outlook', 'through', 'Q4', '2025']
--- Regex social-aware: ['The', 'Reserve', 'Bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points', 'on', 'Tuesday', 'citing', 'persistent', 'inflation', 'Analysts', 'expect', 'a', 'cautious', 'outlook', 'through', 'Q4', '2025']

NLTK:
Sentence: ['The Reserve Bank raised interest rates by 25 basis points on Tuesday, citing persistent inflation.', 'Analysts expect a cautious outlook through Q4 2025.']
Word: ['The', 'Reserve', 'Bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points', 'on', 'Tuesday', 'citing', 'persistent', 'inflation', 'Analysts', 'expect', 'a', 'cautious', 'outlook', 'through', 'Q4', '2025', '.']
Treebank: ['The', 'Reserve', 'Bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points', 'on', 'Tuesday', 'citing', 'persistent', 'inflation.', 'Analysts', 'expect', 'a', 'cautious', 'outlook', 'through', 'Q4', '2025', '.']
Tweet: ['the', 'reserve', 'bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points', 'on', 'tuesday', 'citing', 'persistent', 'inflation', 'analysts', 'expect', 'a', 'cautious', 'outlook', 'through', 'q4', '2025', '.']
Regex: ['The', 'Reserve', 'Bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points', 'on', 'Tuesday', 'citing', 'persistent', 'inflation', 'Analysts', 'expect', 'a', 'cautious', 'outlook', 'through', 'Q4', '2025']
MWE: ['The', 'Reserve', 'Bank', 'raised', 'interest_rates', 'by', '25', 'basis', 'points', 'on', 'Tuesday', 'citing', 'persistent', 'inflation', 'Analysts', 'expect', 'a', 'cautious', 'outlook', 'through', 'Q4', '2025', '.']

TextBlob:
Sentences: ['The Reserve Bank raised interest rates by 25 basis points on Tuesday, citing persistent inflation.', 'Analysts expect a cautious outlook through Q4 2025.']
Words: ['The', 'Reserve', 'Bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points', 'on', 'Tuesday', 'citing', 'persistent', 'inflation', 'Analysts', 'expect', 'a', 'cautious', 'outlook', 'through', 'Q4', '2025']

```

```

spaCy:
Sentences: ['The Reserve Bank raised interest rates by 25 basis points on Tuesday, citing persistent inflation.', 'Analysts expect a cautious outlook through Q4 2025.']
Tokens: ['The', 'Reserve', 'Bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points', 'on', 'Tuesday', 'citing', 'persistent', 'inflation', 'Analysts', 'expect', 'a', 'cautious', 'outlook', 'through', 'Q4', '2025', '.']

Gensim: ['the', 'reserve', 'bank', 'raised', 'interest', 'rates', 'by', 'basis', 'points', 'on', 'tuesday', 'citing', 'persistent', 'inflation', 'analysts', 'expect', 'a', 'cautious', 'outlook', 'through']
Keras: ['the', 'reserve', 'bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points', 'on', 'tuesday', 'citing', 'persistent', 'inflation', 'analysts', 'expect', 'a', 'cautious', 'outlook', 'through', 'q4', '2025']

=== Observations: News ===
NLTK word_tokenize: ['The', 'Reserve', 'Bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points']
TweetTokenizer: ['the', 'reserve', 'bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points']
spaCy: ['The', 'Reserve', 'Bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points']
Gensim: ['the', 'reserve', 'bank', 'raised', 'interest', 'rates', 'by', 'basis', 'points', 'on']
Keras: ['the', 'reserve', 'bank', 'raised', 'interest', 'rates', 'by', '25', 'basis', 'points']

```

```

===== General =====
--- Python split: ['But', 'I\'m', 'glad', 'you\'ll', 'see', 'me', 'as', 'I', 'am.', 'Above', 'all', 'I', 'wouldn\'t', 'want', 'people', 'to', 'think', 'that', 'I', 'want', 'to', 'prove', 'anything', 'I', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', 'I', 'have', 'that', 'right', 'haven\'t', 'I?', '-1', 'Leo', 'Tolstoy']
--- Regex \w+: ['But', 'I', 'm', 'glad', 'you', 'll', 'see', 'me', 'as', 'I', 'am', 'Above', 'all', 'I', 'wouldn', 't', 'want', 'people', 'to', 'think', 'that', 'I', 'want', 'to', 'prove', 'anything', 'I', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', 'I', 'have', 'that', 'right', 'haven', 't', 'I', 'Leo', 'Tolstoy']
--- Regex social-aware: ['But', 'I\'m', 'glad', 'you\'ll', 'see', 'me', 'as', 'I', 'am', 'Above', 'all', 'I', 'wouldn\'t', 'want', 'people', 'to', 'think', 'that', 'I', 'want', 'to', 'prove', 'anything', 'I', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', 'I', 'have', 'that', 'right', 'haven\'t', 'I', 'Leo', 'Tolstoy']

```



```

textBlob:
Sentences: ["But I'm glad you'll see me as I am.", "Above all, I wouldn't want people to think that I want to prove anything.", 'I just want to live; to cause no e
vil to anyone but myself.', "I have that right, haven't I?", '- Leo Tolstoy']
Words: ['But', 'I', 'm', 'glad', 'you', 'll', 'see', 'me', 'as', 'I', 'am', 'Above', 'all', 'I', 'wouldn', 't', 'want', 'people', 'to', 'think', 'that', 'I', 'w
ant', 'to', 'prove', 'anything', 'I', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', 'I', 'have', 'that', 'right', 'ha
ve', 'n't', 'I', '-', 'Leo', 'Tolstoy']

spaCy:
Sentences: ["But I'm glad you'll see me as I am.", "Above all, I wouldn't want people to think that I want to prove anything.", 'I just want to live; to cause no e
vil to anyone but myself.', "I have that right, haven't I?", '- Leo Tolstoy']
Tokens: ['But', 'I', 'm', 'glad', 'you', 'll', 'see', 'me', 'as', 'I', 'am', '.', 'Above', 'all', '.', 'I', 'wouldn', 't', 'want', 'people', 'to', 'think', 'tha
t', 'I', 'want', 'to', 'prove', 'anything', '.', 'I', 'just', 'want', 'to', 'live', ';', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', '.', 'I', 'h
ave', 'that', 'right', ',', 'have', 'n't', 'I', 'p', '-', 'Leo', 'Tolstoy']

Gensim:
['but', 'glad', 'you', 'll', 'see', 'me', 'as', 'am', 'above', 'all', 'wouldn', 'want', 'people', 'to', 'think', 'that', 'want', 'Words: ['But', 'I', 'm', 'glad', 'you', 'll', 'see', 'me', 'as', 'I', 'am', 'Above', 'all', 'I', 'wouldn', 't', 'want', 'people', 'to', 'think', 'that', 'I', 'want', 'to', 'prove', 'anyth
ing', 'I', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', 'I', 'have', 'that', 'right', 'have', 'n't', 'I', '-', 'Leo', 'Tolstoy']

spaCy:
Sentences: ["But I'm glad you'll see me as I am.", "Above all, I wouldn't want people to think that I want to prove anything.", 'I just want to live; to cause no e
vil to anyone but myself.', "I have that right, haven't I?", '- Leo Tolstoy']
Tokens: ['But', 'I', 'm', 'glad', 'you', 'll', 'see', 'me', 'as', 'I', 'am', '.', 'Above', 'all', '.', 'I', 'wouldn', 't', 'want', 'people', 'to', 'think', 'tha
t', 'I', 'want', 'to', 'prove', 'anything', '.', 'I', 'just', 'want', 'to', 'live', ';', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', '.', 'I', 'h
ave', 'that', 'right', ',', 'have', 'n't', 'I', 'p', '-', 'Leo', 'Tolstoy']

```

Gensim: ['but', 'glad', 'you', 'll', 'see', 'me', 'as', 'am', 'above', 'all', 'wouldn', 'want', 'people', 'to', 'think', 'that', 'want', 'I', 'want', 'to', 'prove', 'anything', 'I', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', 'I', 'have', 'that', 'right', 'have', 'n't', 'I', 'Leo', 'Tolstoy']

spaCy:

Sentences: ["But I'm glad you'll see me as I am.", "Above all, I wouldn't want people to think that I want to prove anything.", "I just want to live; to cause no evil to anyone but myself.", "I have that right, haven't I?"] - Leo Tolstoy]

Tokens: ['But', 'I', 'll', 'see', 'me', 'as', 'I', 'am', 'above', 'all', 'I', 'would', 'n't', 'want', 'people', 'to', 'think', 'that', 'I', 'want', 'to', 'prove', 'anything', 'I', 'I', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', 'I', 'have', 'that', 'right', 'I', 'have', 'n't', 'I', 'Leo', 'Tolstoy']

Gensim: ['but', 'glad', 'you', 'll', 'see', 'me', 'as', 'am', 'above', 'all', 'wouldn', 'want', 'people', 'to', 'think', 'that', 'want', 'Sentences: ["But I'm glad you'll see me as I am.", "Above all, I wouldn't want people to think that I want to prove anything.", "I just want to live; to cause no evil to anyone but myself.", "I have that right, haven't I?"] - Leo Tolstoy]

Tokens: ['But', 'I', 'll', 'see', 'me', 'as', 'I', 'am', 'above', 'all', 'I', 'would', 'n't', 'want', 'people', 'to', 'think', 'that', 'I', 'want', 'to', 'prove', 'anything', 'I', 'I', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', 'I', 'have', 'that', 'right', 'I', 'have', 'n't', 'I', 'Leo', 'Tolstoy']

Gensim: ['but', 'glad', 'you', 'll', 'see', 'me', 'as', 'am', 'above', 'all', 'wouldn', 'want', 'people', 'to', 'think', 'that', 'want', 'people', 'to', 'think', 'that', 'I', 'want', 'to', 'prove', 'anything', 'I', 'I', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', 'I', 'have', 'that', 'right', 'I', 'have', 'n't', 'I', 'Leo', 'Tolstoy']

Gensim: ['but', 'glad', 'you', 'll', 'see', 'me', 'as', 'am', 'above', 'all', 'wouldn', 'want', 'people', 'to', 'think', 'that', 'want', 'evil', 'to', 'anyone', 'but', 'myself', 'I', 'have', 'that', 'right', 'I', 'have', 'n't', 'I', 'Leo', 'Tolstoy']

```
Gensim: ['but', 'glad', 'you', 'll', 'see', 'me', 'as', 'am', 'above', 'all', 'wouldn', 'want', 'people', 'to', 'think', 'that', 'want', 'Gensim: ['but', 'glad', 'yo
u', 'll', 'see', 'me', 'as', 'am', 'above', 'all', 'wouldn', 'want', 'people', 'to', 'think', 'that', 'want', 'to', 'prove', 'anything', 'just', 'want', 'to', 'liv
e', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'myself', 'have', 'that', 'right', 'haven', 'leo', 'tolstoy']
Keras: ['but', 'i'm', 'glad', 'you'll', 'see', 'me', 'as', 'i', 'am', 'above', 'all', 'i', 'wouldn't', 'want', 'people', 'to', 'think', 'that', 'i', 'want', 'to',
'prove', 'anything', 'i', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'right', 'haven', 'leo', 'tolstoy']
Keras: ['but', 'i'm', 'glad', 'you'll', 'see', 'me', 'as', 'i', 'am', 'above', 'all', 'i', 'wouldn't', 'want', 'people', 'to', 'think', 'that', 'i', 'want', 'to',
'prove', 'anything', 'i', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'Keras: ['but', 'i'm', 'glad', 'you'll', 'see', 'me',
'as', 'i', 'am', 'above', 'all', 'i', 'wouldn't', 'want', 'people', 'to', 'think', 'that', 'i', 'want', 'to', 'prove', 'anything', 'i', 'just', 'want', 'to', 'live'
, 'to', 'cause', 'no', 'evil', 'to', 'anyone', 'but', 'that', 'i', 'want', 'to', 'prove', 'anything', 'i', 'just', 'want', 'to', 'live', 'to', 'cause', 'no', 'evi
l', 'to', 'anyone', 'but', 'myself', 'i', 'have', 'that', 'right', 'haven't', 'i', '-', 'leo', 'tolstoy']

=== Observations: General ===
NLTK word_tokenize: ['But', 'I', 'i'm', 'glad', 'you', 'll', 'see', 'me', 'as', 'I']
TweetTokenizer: ['but', 'i'm', 'glad', 'you'll', 'see', 'me', 'as', 'i', 'am', '.']
spaCy: ['But', 'I', 'i'm', 'glad', 'you', 'll', 'see', 'me', 'as', 'I']
Gensim: ['but', 'glad', 'you', 'll', 'see', 'me', 'as', 'am', 'above', 'all']
Keras: ['but', 'i'm', 'glad', 'you'll', 'see', 'me', 'as', 'i', 'am', 'above']

RegexTokenizer (w+): ['The', 'price', 'of', 'the', 'car', 'is', '50', '000', 'dollars', 'Model', 'X', 'isn', 't', 'cheap']
RegexTokenizer (keep apostrophes): ['The', 'price', 'of', 'the', 'car', 'is', 'dollars', 'Model', 'X', 'isn't', 'cheap']
RegexTokenizer (numbers with commas/decimals): ['50,000']
```

Post Lab Question-Answers:**1. Explain the tokenization of regular expression using Regexp tokenizer from NLTK with the help of an example**

Ans: The RegexpTokenizer in NLTK uses regular expressions to tokenize the text based on custom patterns. This allows more flexibility compared to default tokenizers, especially when dealing with specific rules for tokenizing text.

```
import nltk
from nltk.tokenize import RegexpTokenizer

# Example text
text = "The price of the car is 50000 dollars."

# Define a regular expression pattern to tokenize words
tokenizer = RegexpTokenizer(r'\w+')

# Apply the tokenizer
tokens = tokenizer.tokenize(text)

print(tokens)
```

- `r'\w+'` is a regular expression pattern that matches sequences of alphanumeric characters (including underscores).
- The output will be: `['The', 'price', 'of', 'the', 'car', 'is', '50000', 'dollars']`.
- The regular expression ensures that punctuation and other non-word characters are excluded.

This method gives fine-grained control over tokenization and can be tailored to the structure of the text you are processing.

Outcomes: CO1 – Understand fundamentals of NLP

Conclusion (based on the Results and Outcomes achieved):

This experiment explored various tokenization methods using different Python libraries, including NLTK, TextBlob, spaCy, and Keras. Each library provides unique features suited to different types of text data, such as tweets, news articles, and general sentences. By comparing these methods, we observed that the choice of tokenizer significantly impacts how text is processed, particularly in handling punctuation, contractions, and special characters like emojis. Understanding these differences is crucial for selecting the appropriate tokenization approach in natural language processing tasks.

References:

Books/ Journals/ Websites:

1. Allen.James, Natural Language Understanding, Benjamin Cumming, Second Edition, 1995
 2. Jurafsky, Dan and Martin, James, Speech and Language Processing, Prentice Hall, 2008
 3. Palash Goyal, Karan Jain, Sumit Pandey, Deep Learning for Natural Language Processing: Creating Neural Networks with Python, Apress, 2018
-